R1.01 - Initiation au Développement : Variables, Types de Données, Allocation Mémoire et Structures Conditionnelles

Thème principal : Bases de l'algorithmique, typage stricts, représentation mémoire et structures de contrôle conditionnelles

Synthèse exécutive : Ce cours pose les fondations fondamentales du développement logiciel en R1.01. L'objectif est de comprendre comment un ordinateur manipule la mémoire vive (RAM) à travers le concept de variable, pourquoi le typage rigoureux des données est indispensable pour éviter les dépassements de capacité (overflow), et comment piloter l'exécution d'un programme via des structures conditionnelles (SI... ALORS... SINON). Une attention particulière est portée à l'exécution pas à pas en mémoire et aux pièges fréquents en évaluation de BUT Informatique.

1. Le concept de Variable et le Modèle Mémoire (RAM)

Pour qu'un programme puisse traiter des informations, il doit pouvoir les conserver temporairement en mémoire vive (RAM). Une variable constitue une abstraction haut niveau d'un emplacement mémoire. Elle se caractérise par trois éléments indissociables : 1) Son NOM (ou identificateur), utilisé par le développeur dans le code. 2) Son TYPE, qui indique au compilateur la quantité exacte d'octets à réserver (ex: 4 octets pour un entier 'int' standard) et la façon d'interpréter les bits. 3) Sa VALEUR, qui représente le contenu binaire actuellement stocké dans cet espace. Lors d'une affectation (ex: 'a = 5'), l'ordinateur ne crée pas un nouvel objet abstrait : il va à l'adresse mémoire associée à 'a' et y enregistre la valeur binaire correspondant à 5. Si l'on réaffecte 'a = 12', l'ancienne valeur 5 est définitivement effacée et remplacée.

- Variable : Espace réservé dans la mémoire RAM identifié par une adresse unique et associé à un nom symbolique.
- Typage : Définition précise de la taille en octets et de la nature de la donnée (entier, flottant, caractère, booléen).
- Affectation : Action de stocker une valeur dans la case mémoire d'une variable en écrasant son ancien contenu.

Cas concret enseignant : *Le professeur a utilisé l'analogie du casier dans un vestiaire d'IUT : chaque casier a un numéro fixe (l'adresse mémoire). Vous collez une étiquette sur le casier avec écrit 'age' (le nom de la variable). La taille du casier détermine ce qu'on peut mettre dedans (le type). Si vous mettez un sac rouge dedans (valeur 20), puis plus tard vous le remplacez par un sac bleu (valeur 21), le sac rouge n'existe plus dans le casier. On ne peut pas mettre un sac d'une taille trop grande sous peine de faire déborder le casier (overflow).*

2. Les Types Primitifs et la Gestion de la Capacité

Le choix du type est crucial en développement algorithmique. Un entier 'int' codé sur 32 bits permet de représenter des valeurs comprises entre -2 147 483 648 et +2 147 483 647. Dépasser cette limite provoque une erreur d'overflow où le nombre repasse dans les négatifs. Les types flottants ('float' sur 32 bits, 'double' sur 64 bits) ne stockent pas des valeurs exactes mais des approximations dues à la représentation en base 2. Il est donc fondamentalement incorrect de tester l'égalité stricte (==) entre deux nombres flottants dans un test conditionnel.

- Entiers (int, long) : Nombres sans virgule, signés ou non signés.
- Flottants (float, double) : Nombres à virgule avec précision limitée (norme IEEE 754).
- Booléens (bool) : État logique VRAI (1) ou FAUX (0).
- Conversion de type (Cast) : Transformation explicite ou implicite d'un type vers un autre.

Cas concret enseignant : *L'enseignant a rappelé le fameux bogue de la fusée Ariane 5 en 1996 : la conversion d'un nombre flottant 64 bits en un entier 16 bits a provoqué un dépassement de capacité (overflow), entraînant la destruction de la fusée en plein vol. En cours, il a insisté : 'Ne choisissez jamais le type float par réflexe pour des montants financiers, utilisez des entiers en centimes'.*

3. Structures Conditionnelles et Évaluation Logique

Par défaut, les instructions d'un programme s'exécutent de manière séquentielle, ligne par ligne. Les structures conditionnelles permettent d'introduire de la prise de décision. Le bloc SI évalue une expression booléenne. Si le résultat est VRAI (True), le bloc d'instructions associé est exécuté. Si le résultat est FAUX (False), l'exécution saute au bloc SINON SI ou SINON. Les opérateurs logiques permettent de combiner plusieurs conditions. L'évaluation paresseuse est un mécanisme fondamental : dans une expression '(A && B)', si A est FAUX, l'ordinateur n'évalue même pas B car le résultat sera impérativement FAUX. De même, pour '(A || B)', si A est VRAI, B n'est pas évalué.

- Structure SI / SINON SI / SINON : Branchement conditionnel modifiant le flux d'exécution.
- Opérateurs Logiques : ET (&&), OU (||), NON (!).
- Évaluation paresseuse (Short-circuit evaluation) : Arrêt de l'évaluation dès que le résultat global est garanti.

Cas concret enseignant : *En cours, un exemple concret a été donné avec un contrôle d'accès : 'SI (utilisateur_existe ET mot_de_passe_correct)'. Si 'utilisateur_existe' est FAUX, le système ne va même pas vérifier le mot de passe en base de données. Cela évite des plantages (ex: tentative de lecture d'une variable nulle) et optimise les performances du programme.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi 0.1 + 0.2 n'est pas exactement égal à 0.3 quand on l'affiche en programmation ?
Réponse du professeur : Parce que les ordinateurs codent les nombres en binaire (base 2). Tout comme 1/3 ne peut pas s'écrire de manière finie en base 10 (0.3333...), 0.1 ne s'écrit pas de façon finie en binaire. Il y a donc une légère imprécision d'arrondi. Pour comparer des flottants, on vérifie que la différence absolue entre les deux nombres est inférieure à une toute petite valeur epsilon (ex: |a - b| < 0.0001).

Question étudiant : Est-ce qu'une variable déclarée dans un bloc SI existe toujours après la fin du bloc SINON ?
Réponse du professeur : Non. C'est la notion de portée des variables (scope). Une variable déclarée à l'intérieur d'un bloc de code (entre accolades ou identée dans ce bloc) est détruite de la mémoire RAM dès la sortie de ce bloc. Si vous en avez besoin plus tard, vous devez la déclarer avant la condition.

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme d'un programme qui demande l'âge d'un utilisateur et son statut d'étudiant (Booleen), puis calcule le prix d'un billet de cinéma sachant que : Le tarif plein est de 10 €. Les mineurs (< 18 ans) paient 6 €. Les étudiants majeurs bénéficient d'un tarif réduit à 7.50 €. Les seniors (>= 65 ans) paient 5 €.

Résolution pas à pas :
Étape 1 : Déclarer les variables nécessaires : 'age' (Entier), 'estEtudiant' (Booléen), 'prix' (Flottant). Étape 2 : Saisir les valeurs d'entrée 'age' et 'estEtudiant'. Étape 3 : Structurer la condition imbriquée ou en chaîne : - SI (age < 18) ALORS prix = 6.00 - SINON SI (age >= 65) ALORS prix = 5.00 - SINON SI (estEtudiant == VRAI) ALORS prix = 7.50 - SINON prix = 10.00 Étape 4 : Afficher le prix calculé.

Résultat : Pour un utilisateur de 20 ans avec statut étudiant = VRAI, le programme évalue age < 18 (FAUX), age >= 65 (FAUX), estEtudiant == VRAI (VRAI) -> Le prix attribué est 7.50 €. L'ordre des conditions garantit qu'un mineur étudiant paiera 6 € (tarif le plus avantageux).

■ Auto-Évaluation Flash d'Examen

Q : Que se produit-il si l'on tente d'affecter la valeur 300 à une variable entente stockée sur un octet non signé (uint8_t) ?

R : Un dépassement de capacité (overflow). La valeur maximale sur 8 bits étant 255 (2^8 - 1), 300 sera tronqué et la variable va 'enrouler' pour stocker 300 - 256 = 44.

Q : Dans une condition 'SI (A || B)', quelle est l'utilité de l'évaluation court-circuit si A est VRAI ?

R : L'expression B n'est pas évaluée du tout, ce qui économise du temps de calcul et évite d'exécuter une instruction potentiellement dangereuse dans B.

Q : Quelle est l'erreur principale à éviter lors de la comparaison de deux variables de type float ?

R : Utiliser l'opérateur d'égalité stricte (==). Il faut utiliser une comparaison par tolérance avec une valeur epsilon : fabs(a - b) < EPSILON.